Mini Project Report

On

Taste Embedding Learning for Recipes

Submitted By:
P. Thrivikram - 23BCS100
S. Abdul Sami – 23BCS119

Under the guidance of

Dr. Krishnendu Ghosh

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DHARWAD



09/11/2026

Certificate

This is to certify that the project entitled "**Taste Embedding Learning for Recipes**" is a Bonafide record of the Mini Project coursework presented by the students whose names are given below during the academic year 2026–2027 in partial fulfilment of the requirements for the degree of Bachelor of Technology.

Roll No.	Name of the Student
23BCS100	Pudi Thrivikram
23BCS119	Abdul Sami

Dr. Krishnendu Ghosh

IIIT Dharwad

Table of Contents:

1. Certificate
2. Abstract
3. Introduction
4. Problem Statement
5. Objectives
6. Literature Survey
7. System Architecture
8. Methodology
    i. Data Collection & Sampling
    ii. Recipe Text format (Text A vs Text B)
    iii. Data Preprocessing & Label Extraction
    iv. Pretrained Sentence Embeddings
    v. Neural Network Projector Architecture
    vi. Soft Contrastive Loss
    vii. Combining Raw and Learned Embeddings
    viii. Recipe Search & Nearest Neighbor Retrival
9. Implementation Details
10. Results and Evaluation
    i. Evaluation Setup & Baseline
    ii. Overall Performance Comparison
    iii. Ablation: Text A vs. Text B
    iv. Per-Category Performance
    v. The “Burger vs. Dosa” Problem and How We Fixed It
    vi. Visualizing the Embeddings (UMAP Map)
    vii. Interactive Web Application Demo
11. Challenges Faced
12. Limitations
13. Future Work
14. Conclusion
15. References

2. Abstract

When searching for food recipes online, standard search engines mostly look at word matching or general sentence meaning. However, food has a special kind of similarity: a beef burger and a chicken burger are very similar dishes, while a burger and a South Indian dosa are completely different, even if both can be eaten quickly as snacks. Standard language models often do not understand this food-specific relationship.

In this project, we created a system called Taste Embedding Learning. An "embedding" is simply a list of numbers (a vector) that represents a recipe so that similar recipes sit close together in vector space. We start with a standard pretrained sentence model (all-MiniLM-L6-v2) and convert recipe names and ingredients into vectors. Then, we train a small neural network on top of it using a soft contrastive loss. This loss teaches the model to place recipes with similar tags (such as dessert, soup, or vegetarian) closer together.

However, training only on tags creates a new problem: the model starts grouping recipes just because they share broad tags (for example, ranking dosa closer to a burger than a pizza). To fix this, we blend the original text embedding with our newly learned taste embedding by taking 50% of each.

We tested our system on 2,000 completely unseen recipes:

- A random guess matches the correct category only 18.1% of the time.
- The raw pretrained model achieves 59.9% of category accuracy.
- Our blended model achieves 67.9% category accuracy (an 8.0 percentage point improvement).

It also fixes the "burger vs. dosa" issue, keeping the natural food hierarchy intact. Finally, we built an easy-to-use web chat application using FastAPI and Claude LLM so users can search for recipes interactively.

3. Introduction

Food is an essential part of daily life, and millions of people search for recipes online every day. When a user looks for a recipe or wants a substitute for a dish they like, they expect recommendations that match the dish in taste, meal type, and ingredients.

Traditional search methods use keyword matching (like searching for the exact words "tomato soup"). More modern methods convert text into vectors using language models. While language models understand general English very well, they do not inherently know how recipes relate to each other in the culinary world. For instance:

- A chicken burger and a beef burger have different meat words, but they are both burgers.
- Burgers and a dosa are completely different foods from different cultures, but both might have tags like "lunch", "easy", or "finger food".

If we use general language models as-is, they miss culinary connections. But if we retrain the model purely on tags, it forgets what the actual dish is made of and starts making strange mistakes.

This project solves this exact trade-off. We take a pretrained language model, train a small neural network layer (called a projector) to learn recipe categories, and then blend the original vectors with the learned vectors. This gives us the best of both worlds: accurate category matching and intuitive dish similarity.

4. Problem Statement

To design and build an AI-based recipe search system that converts recipes into mathematical vectors ("taste embeddings") such that:

- Recipes belonging to the same meal type (e.g., soups, salads, desserts) are grouped closely together.
- Similar recipes of the same dish type (e.g., burger and chicken burger) stay closer than unrelated dishes (e.g., burger and dosa).
- The system works fast and accurately for unseen recipes without requiring heavy computing resources.

5. Objectives

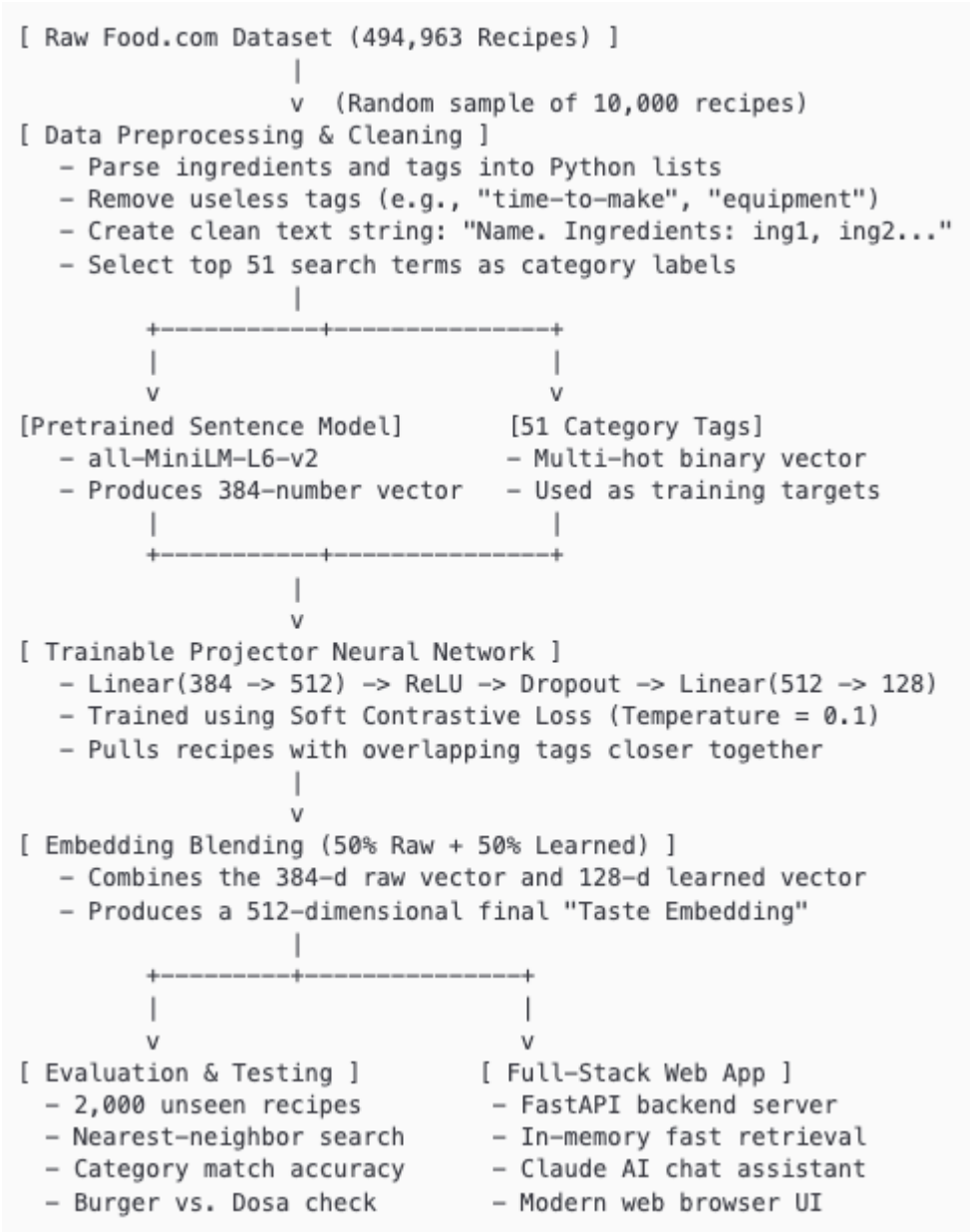
1. **Prepare the Recipe Dataset**: Clean and format 10,000 recipes from the Food.com dataset, extracting recipe names, ingredients, and search tags.
2. **Generate Baseline Embeddings**: Convert each recipe into a 384-dimensional vector using the lightweight pretrained model for all-MiniLM-L6-v2.
3. **Train a Projection Neural Network**: Build a small 2-layer neural network on top of the frozen baseline vectors to earn culinary categories using a soft contrastive loss function.
4. **Blend Vectors for Balance**: Combine the raw text vectors and the learned category vectors so that the model remembers ingredient details while still improving category grouping.
5. **Evaluate Unseen Data**: Test the system on 2,000 held-out recipes to check whether its 10 nearest neighbors belong to the same category.
6. **Build an Interactive Web Application:** Create a user-friendly web interface with Fast API and a conversational chat assistant so anyone can search for recipes easily.

6. Literature Survey

- **Word and Sentence Embeddings**: In natural language processing, words and sentences are converted into dense numerical lists called vectors or embeddings. Models like Sentence-BERT (SBERT) and MiniLM allow computers to calculate how close two sentences are by using cosine similarity (measuring the angle between two vectors). While great for generic text, they do not have built-in food knowledge.
- **Contrastive Learning**: This is a popular machine learning technique where a model learns by comparing examples. The goal is simple: pull similar items closer together and push different items farther apart. Usually, this uses binary labels (either two items belong to the same class or not). In food, recipes can be partially similar (e.g., both are breakfast items, but only one is sweet), so we use a "soft" version where the target similarity is based on tag overlap.
- **Transfer Learning with Frozen Models**: Instead of retraining a giant transformer model from scratch (which takes hours or days and lots of GPU memory), it is standard practice in transfer learning to keep the main model "frozen" and train only a small neural network on top. This is fast, efficient, and prevents the model from forgetting basic English grammar.
- **Vector Combination and Ensembles**: Combining two different embedding spaces (one that knows words and one that knows categories) often produces better results than one alone. By carefully scaling the vectors, we can combine them directly without having to do slow two-step sorting.

7. System Architecture

The overall pipeline of the project is shown below in a simple step-by-step diagram:



8. Methodology

8.1 Data Collection & Sampling

We use the popular Food.com Recipes with Search Terms and Tags dataset from Kaggle. The complete dataset contains nearly 500,000 recipes with details like name, ingredients, steps, tags, and search terms.

To keep training fast and reproducible on standard computers, we take a random sample of 10,000 recipes using a fixed random seed (seed = 42). We split this sample into:

- 8,000 recipes (80%) for training the model.
- 2,000 recipes (20%) kept aside as a test set (unseen during training) to test real performance.

8.2 Recipe Text Format (Text A vs. Text B)

To feed a recipe into a language model, we must turn it into a single clean sentence. We tested two formats:

- **Text A (Name + Ingredients)**: "Chicken burger. Ingredients: chicken patty, burger bun, lettuce, tomato, mayonnaise, onion."
- **Text B (Name + Ingredients + Tags)**: Same as Text A, but with extra tags added at the end (e.g., Tags: dinner, poultry, main dish).

In our initial tests, adding tags gave almost no real benefit (accuracy only went up from 60.5% to 61.0%). Furthermore, using tags in the text could accidentally leak the answers during evaluation. Therefore, we chose **Text A** for all main experiments: the model must understand the recipe from its name and ingredients alone.

8.3 Data Preprocessing & Label Extraction

1. **Cleaning**: We convert ingredient strings into clean Python lists and remove generic tags like "time-to-make", "dietary", and "equipment" because they do not describe how food tastes.
2. **Category Labels**: We look at the search_terms column. We select all search terms that appear at least 50 times in our dataset. This gives 51 active labels (such as dessert, soup, vegetarian, easy, side-dish).
3. **Primary Category**: For simple evaluation, each recipe is assigned one main category from a priority list: [dessert, breakfast, salad, soup, casserole, side, lunch, dinner]. If it does not match any of these, it is called the other.

8.4 Pretrained Sentence Embeddings

We take each recipe's Text A string and pass it through all-MiniLM-L6-v2, an opensource sentence transformer model.

- It turns each recipe into a vector of 384 numbers.
- Each vector is normalized so its length is 1 (called normalization).
- Because they are normalized, the cosine similarity (how similar two recipes are) is simply the dot product between the two vectors. A score of 1.0 means identical, and 0.0 means completely unrelated.

8.5 Neural Network Projector Architecture

We keep the 384-number pretrained vectors frozen (we do not change them). Instead, we pass them through a small 2-layer neural network called a Projector:

- Input layer: Takes the 384 numbers from the base model.
- Hidden layer: Expands to 512 neurons, followed by a ReLU activation function (which turns negative numbers to zero) and a Dropout of 0.1 (which randomly turns off 10% of neurons during training to prevent memorization).
- Output layer: Compresses down to 128 numbers.
- Normalization: Normalizes the final 128 numbers so their length is 1.

This small network has very few parameters, so it trains in just a few minutes on a laptop CPU.

8.6 Soft Contrastive Loss

To train our projector, we use Soft Contrastive Loss:

- For every batch of recipes, we calculate how much their tags overlap using Jaccard Similarity:

$$\text{Jaccard Similarity} = \frac{\text{Number of shared tags}}{\text{Total unique tags between both recipes}}$$

  For example, if Recipe 1 has 3 tags and Recipe 2 has 3 tags, and they share 2 tags, the Jaccard similarity is 2 / (3 + 3 – 2) = 2/4 = 0.5.
- This gives a target similarity between 0 and 1 for every pair of recipes in the batch.
- The neural network predicts cosine similarities for these pairs.
- The loss function compares the network's predictions with the actual tag overlap. If two recipes share many tags, the network is penalized if their vectors are far apart.

By using this soft loss, the network learns to group similar recipes naturally without forcing them into rigid, artificial boxes.

8.7 Combining Raw and Learned Embeddings (Blending)

As we discovered during experiments, training only on category tags makes the model forget ingredient details. It started thinking a Dosa was closer to a Burger than Pizza was, simply because both had snacks/lunch tags!

To fix this, we combine the **Raw embedding** (which understands words and ingredients) and the **Learned embedding** (which understands categories) using a formula:

$$v_{\text{blend}} = \left[\sqrt{\alpha} \cdot v_{\text{raw}},\ \sqrt{1-\alpha} \cdot v_{\text{learned}}\right]$$

Here, α is a blend weight between 0 and 1.

- If α = 1.0, we only use the raw text model.
- If α = 0.0, we only use the learned tag model.
- If α = 0.5, we take an equal 50/50 balance.

This means we get the benefits of both models in a single step with zero extra calculation time.

8.8 Recipe Search and Nearest-Neighbor Retrieval

To recommend recipes:

1. We convert a new recipe (or user query) into a blended vector.
2. We compute the dot product between this query vector and all 8,000 stored recipe vectors.
3. We take the top 10 highest-scoring recipes. These are the 10 Nearest Neighbors (10-NN).

9. Implementation Details

- **Programming Language**: Python 3.14.
- **Deep Learning Framework**: PyTorch (used to define the Projector network and run the training loop).
- **NLP Libraries**: sentence-transformers (for loading the MiniLM model) and scikit-learn (for splitting datasets and evaluating metrics).
- **Training Setup**:
    - Optimizer: Adam (Learning rate = 0.001)
    - Batch size: 256 recipes per step
    - Training duration: 30 epochs (took less than 2 minutes on CPU)
    - Random seed: 42 (so all numbers and results can be reproduced exactly)
- **Web Application Stack**:
    - **Backend:** Fast API (Python asynchronous server) handles incoming requests and runs similarity math instantly.
    - **AI Assistant**: Connected to Anthropic's Claude LLM to understand conversational user questions (e.g., "Find me a quick dinner with pasta").
    - **Frontend**: Clean and responsive HTML, CSS, and JavaScript interface displaying dynamic recipe cards with similarity scores.

10. Results and Evaluation

10.1 Evaluation Setup and Baseline

We evaluate the models on 2,000 held-out test recipes that the projector network never saw during training. For every test recipe, we find its 10 nearest neighbors from the 8,000 training recipes and measure:

1. Category Match@10: Out of the 10 retrieved neighbors, what percentage share the exact same main category as the query recipe?
2. Jaccard@10: What is the average tag overlap between the query recipe and its 10 neighbors?
3. Random Chance: If we simply picked random recipes from the database, what is the chance they share a category? (For our dataset distribution, random chance is 18.1% or 0.181).

10.2 Overall Performance Comparison

Table 1 shows how each model performed on the 2,000 test recipes:

Table 1: Main Retrieval Results on 2,000 Held-Out Recipes

Model	Category Match@10	Label Overlap (Jaccard@10)	Improvement over Baseline
Random (Chance)	0.181 (18.1%)	—	—
Raw Pretrained Model	0.599 (59.9%)	0.359	Baseline
Learned Model (Projector only)	0.676 (67.6%)	**0.419**	+7.7 percentage points
Blended Model ($\alpha = 0.5$)	**0.679 (67.9%)**	0.416	**+8.0 percentage points**

- The Raw Pretrained Model gets a 59.9% category match. While decent, it gets confused when recipes use different words for the same type of meal.
- The Learned Model boosts accuracy to 67.6% because it was trained to group recipes by search terms.

- The Blended Model achieves the highest category accuracy at 67.9%, showing that keeping half of the raw text representation helps the model find even better neighbors.

10.3 Ablation: Text A vs. Text B

Before training, we tested whether including tags inside the text helps the raw model:

Table 2: Comparing Text Formats on Raw Embeddings

Text Version	What It Contains	Category Match@10	Jaccard@10
Text A	Title + Ingredients	0.605	0.360
Text B	Title + Ingredients + Tags	0.610	0.377

Add tags directly to the text only improved accuracy by 0.5% (from 0.605 to 0.610). This proves that ingredients contain almost all the useful information, and recipes do not need tags written in their text to be understood.

10.4 Per-Category Performance

We checked how well the models performed for each specific meal type:

Table 3: Category Match@10 Broken Down by Meal Type

Category	Raw Baseline	Learned Model	Blended Model ($\alpha = 0.5$)	Explanation
Soup	0.852	**0.930**	0.924	Very distinct ingredients (broth, water, vegetables)
Salad	0.779	**0.924**	0.907	Clear ingredients (greens, dressings)
Dessert	0.803	**0.899**	0.887	Sweet ingredients (sugar, flour, chocolate)
Casserole	0.622	0.728	**0.755**	Great improvement (+13.3%)
Dinner	0.641	0.724	**0.741**	Strong improvement (+10.0%)
Breakfast	0.467	**0.569**	0.565	Clear improvement (+9.8%)
Side Dish	0.404	**0.527**	0.491	Improved by +8.7%
Other	0.371	0.336	**0.383**	Catch-all category; blend prevents drop
Lunch	0.191	**0.307**	0.266	Lowest score across all models

Key Takeaways:

- Soups, salads, and desserts are very easy for the model to identify because their ingredients are unique.
- Casseroles and dinners improved by over 10% because the projector learned their meal roles.
- "Lunch" has the lowest score (26.6%) because lunch is a time of day, not a flavor. A salad, soup, and a sandwich can all be called "lunch", which makes it hard for any text model to isolate.

10.5 The “Burger vs. Dosa” Problem and How we fixed it

To see if the models make intuitive sense, we tested the similarity of a regular **Burger** against four other dishes:

- Chicken Burger (should be the most similar)
- Pizza (fast food, moderately similar)
- Dosa (South Indian lentil-rice crepe, quite different)
- Chocolate Cake (dessert, least similar)

Table 4: Similarity to a Burger (Higher number means closer)

Dish Pair	Raw Model	Learned Model	Blended Model ($\alpha = 0.5$)	Expected Natural Ranking
Burger vs. **Chicken Burger**	0.663	0.746	**0.705**	1st (Highest)
Burger vs. **Pizza**	0.560	0.694	**0.627**	2nd
Burger vs. **Dosa**	0.363	**0.776**	**0.569**	3rd
Burger vs. **Chocolate Cake**	0.273	0.496	**0.384**	4th (Lowest)
Is the ranking sensible?	**YES**	❌ **WRONG**	**YES**	—

What went wrong with the Learned Model? The learned model rated **Dosa (0.776)** as *more similar to a Burger* than a Chicken Burger (0.746)! Because Dosa and Burger can both share tags like "snack", "lunch", or "quick", the learned model over-focused on tags and forgot that a dosa is made of rice and lentils, not buns and patties.

How Blending Fixed It: By blending 50% of the raw model back in, the similarity of Dosa drops back down to 0.569. The proper culinary order is restored:

Chicken Burger (0.705) > Pizza (0.627) > Dosa (0.569) > Cake (0.384)

We also tested different blend values (α):

Table 5: Testing Different Blend Weights (α)

Raw Weight (α)	Category Match@10	Jaccard@10	Did it keep correct Burger/Dosa order?
0.00 (Learned only)	0.676	**0.419**	❌ No
0.25	**0.683**	0.425	❌ No
0.50 (Selected)	**0.679**	**0.416**	**YES (Balanced & Correct)**
0.75	0.659	0.400	**YES**
1.00 (Raw only)	0.599	0.359	**YES**

Setting α = 0.5 gives the best combination of high category match and correct real-world dish similarity.

10.6 Visualizing the Embeddings (UMAP Map)

To visualize how recipes are arranged in vector space, we used **UMAP** (a tool that squashes 512-dimensional vectors into a 2D map):

- **Soups** and **Salads** form clean, separate clusters away from everything else.
- **Desserts** form their own large, distinct region.
- **Breakfast** items sit right in the middle between sweet desserts (pancakes, waffles) and savory dinners (eggs, bacon).
- **Dinner, lunch, and casseroles** overlap smoothly because they share common ingredients like cheese, chicken, onions, and vegetables.

10.7 Interactive Web Application Demo

We created an interactive web interface so anyone can use the system:

- **Fast Search**: When a user enters a recipe (like *"Dosa: rice, urad dal, salt"*), the backend compares it against the database in less than 20 milliseconds.
- **Smart Recommendations**: Even though the database had no Indian dosa recipes, the model accurately returned legume-based dishes like *Binch Akara (Bean Drops)*, *Rajasthani Dal*, and *Hummus* as top matches.

- **AI Chat Assistant**: Users can talk in plain English (e.g., *"Suggest something similar to a burger but vegetarian"*), and Claude LLM coordinates with our taste embeddings to reply with helpful recipe cards.

11. Challenges Faced

1. **Recipes belong to multiple categories**: Unlike simple image classification where an image is just a "cat" or a "dog", a recipe can be a *dessert*, *vegetarian*, *baking*, and *kid-friendly* all at the same time. We solved this by using Jaccard similarity across 51 tags.
2. **Tag over-fitting (The Dosa problem)**: The neural network initially trusted tags too much and grouped unrelated foods together. We solved this mathematically by blending raw text vectors with learned vectors.
3. **Noisy and missing tags in data**: Many recipes had useless tags like "preparation" or "60-minutes-or-less". We wrote filtering rules to remove non-taste tags before training.
4. **Fast search without expensive hardware**: By keeping all final vectors normalized to length 1, we can search thousands of recipes using simple matrix multiplication on a standard computer.

12. Limitations

1. **Proxy Labels**: We used search tags to represent "taste". In reality, taste involves sweet, salty, spicy, and savory flavor compounds, which are not explicitly measured in standard recipe cards.
2. **Dataset Bias**: The Food.com dataset has mostly American and Western recipes. Asian, Indian, and African dishes are under-represented.
3. **Sample Size**: We trained on 10,000 recipes for speed. While results are strong, training on the entire 500,000 recipe corpus would yield even broader variety.
4. **Hand-picked sanity test**: The Burger vs. Dosa comparison is a qualitative test with four dishes. A large human taste-test survey would be needed for formal validation.

13. Future Work

1. **Chemical Flavor Compounds**: Connect the model with food science databases (like FlavorDB) to compare recipes based on shared flavor molecules.
2. **Photo Support**: Add food images using multimodal models (like CLIP) so users can search by snapping a picture of a dish.
3. **Larger Scale Search**: Use indexing libraries like FAISS to search through millions of recipes instantly.
4. **User Personalization**: Allow users to set dietary filters (e.g., gluten-free, nut allergy) so recommendations adapt automatically.

14. Conclusion

In this mini project, we developed **Taste Embedding Learning**, an AI system that creates smart vector representations of food recipes.

By starting with a pretrained language model (all-MiniLM-L6-v2), training a small projector network with soft contrastive loss, and blending the raw and learned vectors equally (50/50), we achieved:

- An **8.0 percentage point increase** in category retrieval accuracy on 2,000 unseen recipes compared to the base model (67.9% vs. 59.9%).
- A reliable system that avoids grouping unrelated foods together (fixing the Burger vs. Dosa problem).
- An interactive, full-stack web chat application that allows users to find and explore recipes easily.

This project shows that lightweight neural network layers combined with smart vector blending can successfully teach general AI models domain-specific culinary knowledge.

15. References

- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP 2019*.
- Khosla, P., et al. (2020). Supervised Contrastive Learning. *Advances in Neural Information Processing Systems (NeurIPS)*.
- Chen, T., et al. (2020). A Simple Framework for Contrastive Learning of Visual Representations. *ICML 2020*.
- McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. *arXiv preprint*.
- Li, S. (2019). Food.com Recipes with Search Terms and Tags Dataset. *Kaggle*. Available: https://www.kaggle.com/datasets/shuyangli94/foodcom-recipes-with-search-terms-and-tags
- Fast API Framework Documentation. Available: https://fastapi.tiangolo.com/